Coursework 4 Report

Question 1

All 83 ECG wavelet features were normalised with `MinMaxScaler`. KMeans was fitted with three clusters, corresponding to the three ECG classes: N, S, and V. Since cluster labels are arbitrary, predicted clusters were mapped to true class labels before calculating the metrics.

PCA was then applied to retain more than 90% cumulative explained variance. This retained 17 principal components and 90.75% cumulative explained variance.

Method	Features/components	Accuracy	Macro precision	Macro recall	Macro F1
KMeans, all normalised features	83	0.8917	0.8949	0.8917	0.8896
KMeans, PCA features	17	0.7128	0.7138	0.7128	0.6885

Confusion matrix using all normalised features:

True class \ Predicted class	N	S	V
N	599	1	0
S	101	463	36
V	5	52	543

After PCA:

True class \ Predicted class	N	S	V
N	588	10	2
S	56	196	348
V	3	98	499

Using all normalised features gave the better KMeans result. PCA reduced the feature dimension from 83 to 17, but the accuracy dropped from 0.8917 to 0.7128.

Figure 1 shows that the all-feature model identified N and V well, while most errors came from S beats. After PCA, many S beats were assigned to V, which explains the lower performance.

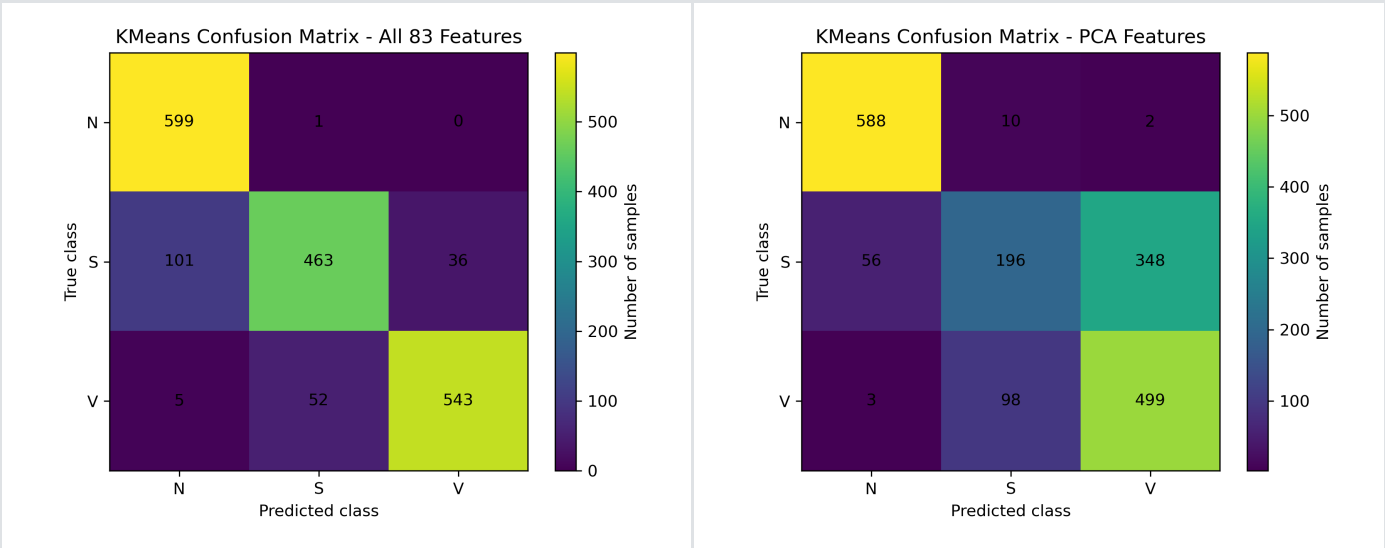

Figure 1. Confusion matrices for KMeans using all 83 normalised features (left) and the PCA-reduced feature set (right).

Figure 2 shows that the 90% explained variance threshold was reached with 17 principal components.

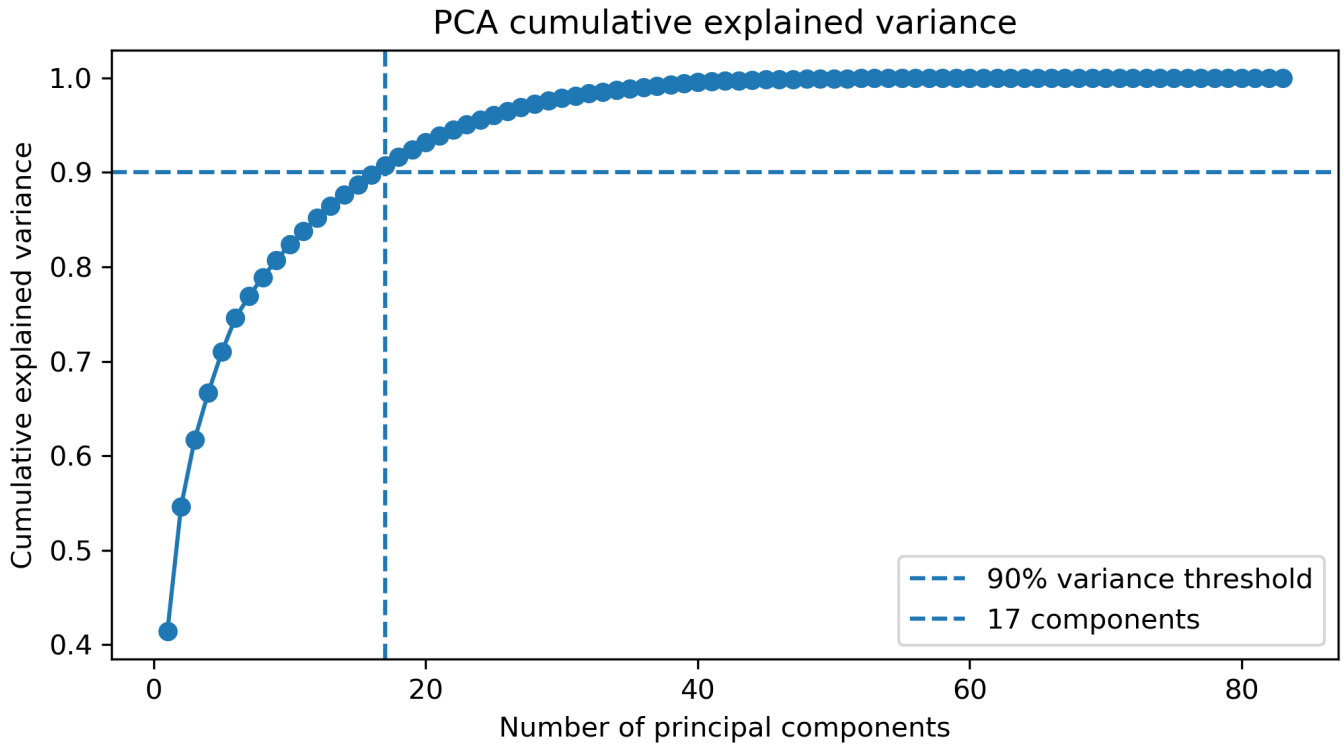

Figure 2. Cumulative explained variance for PCA in Question 1. The selected PCA representation retained more than 90% variance with 17 components.

This indicates that preserving variance does not necessarily preserve class separability, and some discriminative information may have been lost after PCA.

Question 2

The GMM-EM model used three components and full covariance matrices. The model was applied first to all 83 normalised features, then to the PCA representation retaining more than 90% variance.

Method	Features/components	Accuracy	Macro precision	Macro recall	Macro F1
GMM-EM, all normalised features	83	0.9233	0.9232	0.9233	0.9227
GMM-EM, PCA features	17	0.8122	0.8349	0.8122	0.8104

Confusion matrix using all normalised features:

True class \ Predicted class	N	S	V
N	597	3	0
S	41	525	34
V	3	57	540

After PCA:

True class \ Predicted class	N	S	V
N	597	3	0
S	44	449	107
V	178	6	416

The full normalised feature set gave the best GMM-EM performance, with an accuracy of 0.9233 and macro F1-score of 0.9227. After PCA, the accuracy dropped to 0.8122.

Figure 3 shows that the all-feature GMM classified all three classes more reliably. After PCA, many V samples were assigned to N, which reduced the overall accuracy.

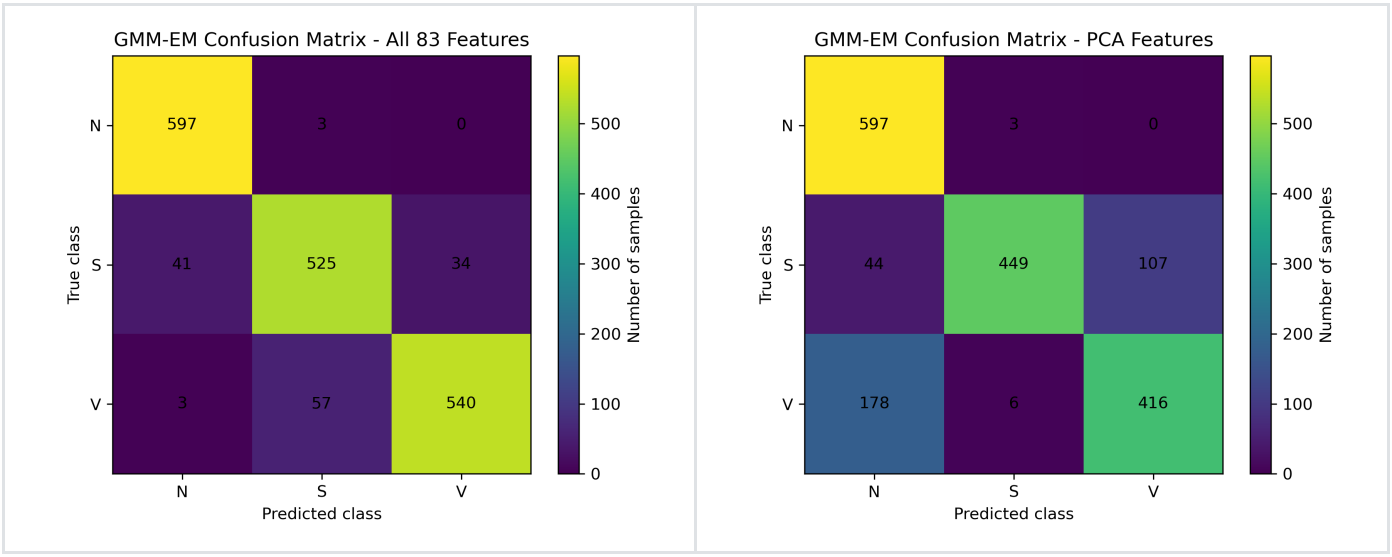

Figure 3. Confusion matrices for GMM-EM using all 83 normalised features (left) and the PCA-reduced feature set (right).

Figure 4 shows that 17 PCA components retained more than 90% cumulative explained variance.

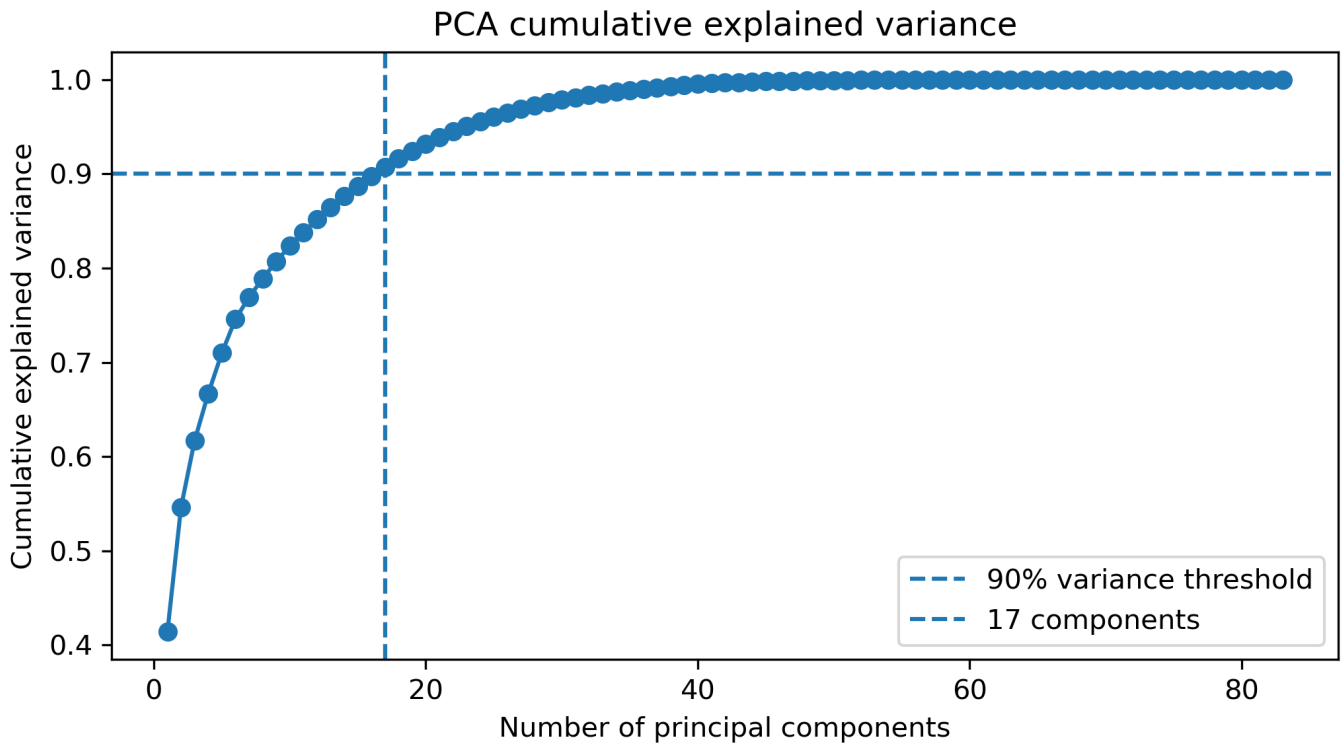

Figure 4. Cumulative explained variance for PCA in Question 2.

This suggests that GMM benefits from the full feature space, and PCA may remove useful distributional information required for accurate clustering.

Question 3

Agglomerative clustering was tested with `average` and `complete` linkage. Features were normalised with `MinMaxScaler`, and the best linkage was selected by accuracy.

Feature representation	Best linkage	Accuracy	Macro precision	Macro recall	Macro F1
All 83 normalised features	complete	0.3622	0.6843	0.3622	0.2258
PCA features, 17 components	complete	0.3394	0.6250	0.3394	0.1799

All-feature linkage comparison:

Linkage	Accuracy	Macro precision	Macro recall	Macro F1
complete	0.3622	0.6843	0.3622	0.2258
average	0.3400	0.6899	0.3400	0.1808

PCA linkage comparison:

Linkage	Accuracy	Macro precision	Macro recall	Macro F1
complete	0.3394	0.6250	0.3394	0.1799
average	0.3350	0.7780	0.3350	0.1702

Agglomerative clustering performed poorly overall. Complete linkage was the best option for both feature representations, but the all-feature representation was slightly better than PCA.

Figure 5 shows that most samples were assigned to the cluster mapped as N. This caused low recall for S and V.

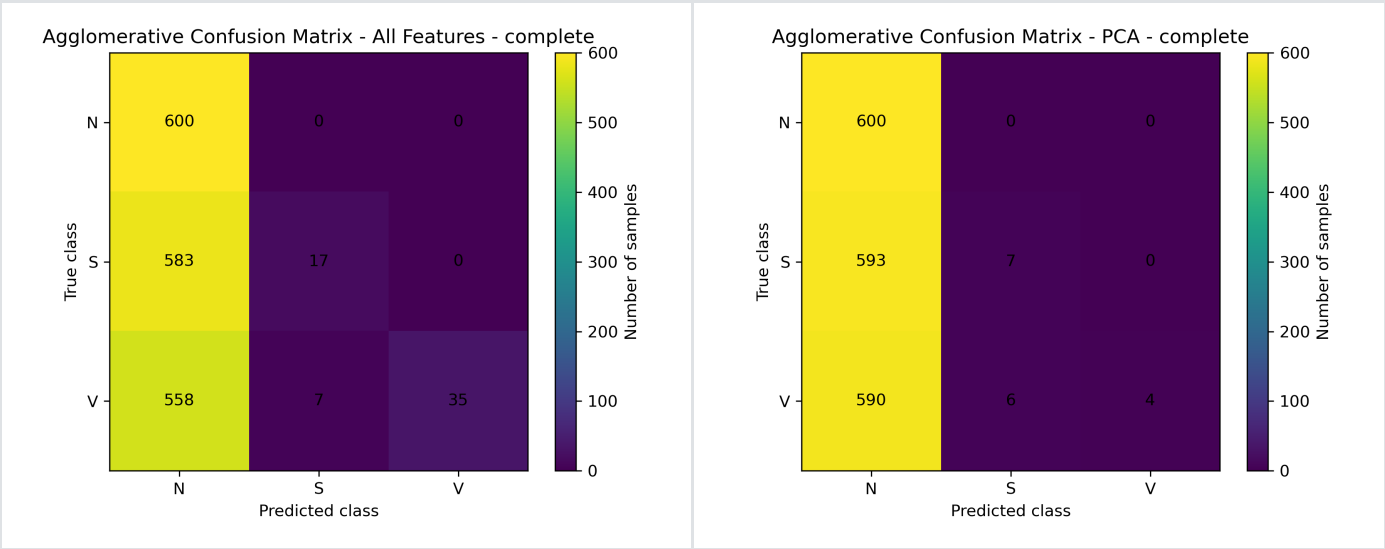

Figure 5. Confusion matrices for Agglomerative clustering using all 83 normalised features with complete linkage (left) and PCA features with complete linkage (right).

Figure 6 shows that PCA retained more than 90% variance with 17 components.

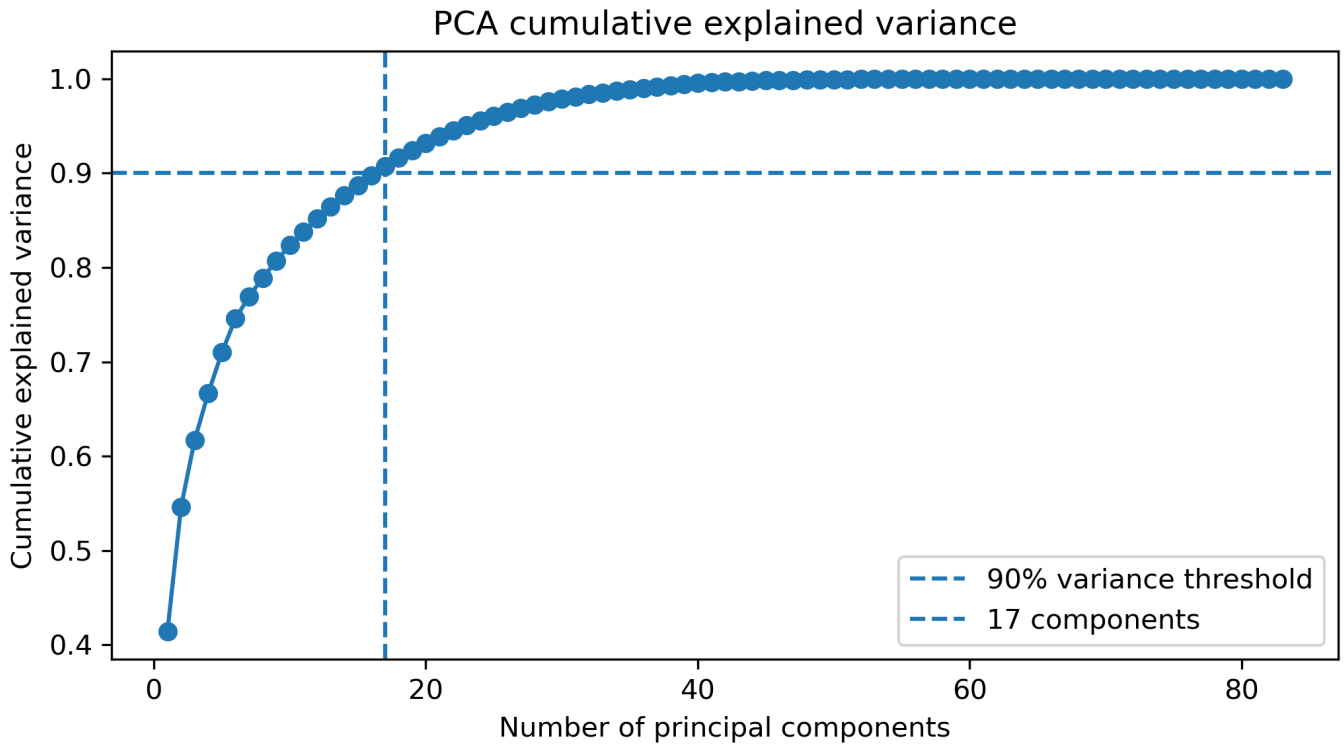

Figure 6. Cumulative explained variance for PCA in Question 3.

Class N was the easiest to cluster, while S was the most difficult, with many samples misclassified. Among the three methods, GMM performed best, followed by KMeans, while Agglomerative clustering performed worst.

Question 4

The first 3000 ECG samples were loaded from `single_ecg_signal.csv`. Stationarity was tested with the Augmented Dickey-Fuller test for the original signal, first difference, square-root transform, and log transform. The first difference signal had the lowest p-value and was selected for modelling.

Signal version	ADF statistic	p-value	Used lags	Stationary at 5%
Original	-11.0742	4.4740e-20	10	True
First difference	-14.8963	1.5305e-27	29	True
Square root	-6.8806	1.4370e-09	28	True
Log	-8.6441	5.3282e-14	14	True

The selected signal was split into the first 1800 samples for training and the remaining 1199 samples for testing. Seasonal decomposition used a period of 360 samples, based on the 360 Hz ECG sampling rate. ACF and PACF suggested `p = 5` and `q = 5`.

Model	Order	RSS	AIC	BIC
MA	(0, 0, 5)	162703.5358	9890.9725	9929.4413
ARMA	(5, 0, 5)	166249.4891	9351.2597	9417.2062
ARIMA	(5, 1, 5)	167556.9663	9344.6001	9405.0449
AR	(5, 0, 0)	170226.4257	9414.1143	9452.5831

The MA(5) model had the lowest RSS on the test set, so it was the best model by the RSS comparison criterion.

Figure 7 shows the original ECG signal, including repeated sharp peaks and changes in amplitude.

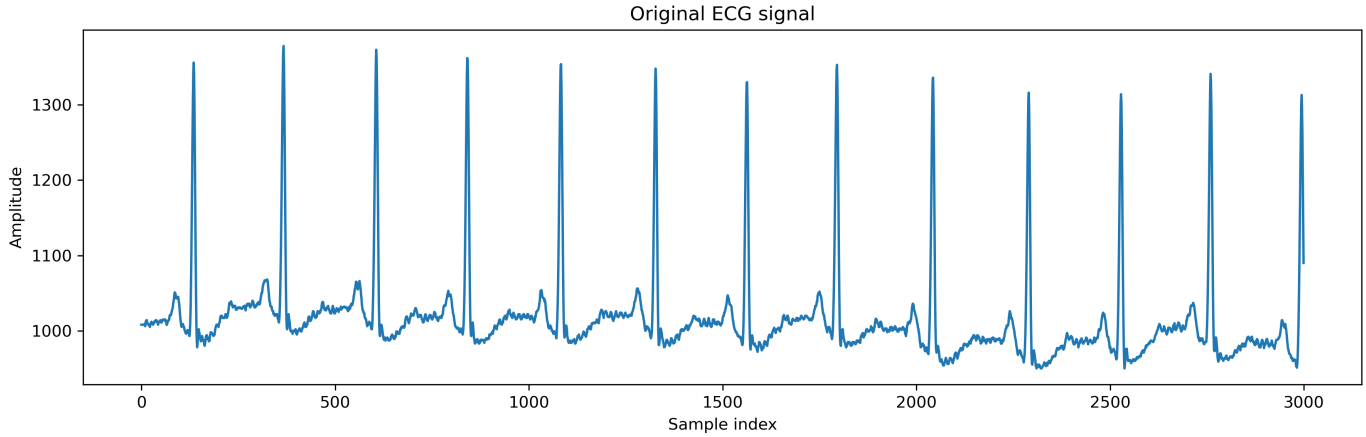

Figure 7. Original ECG signal containing the first 3000 samples.

Figure 8 shows the first-difference signal selected for modelling.

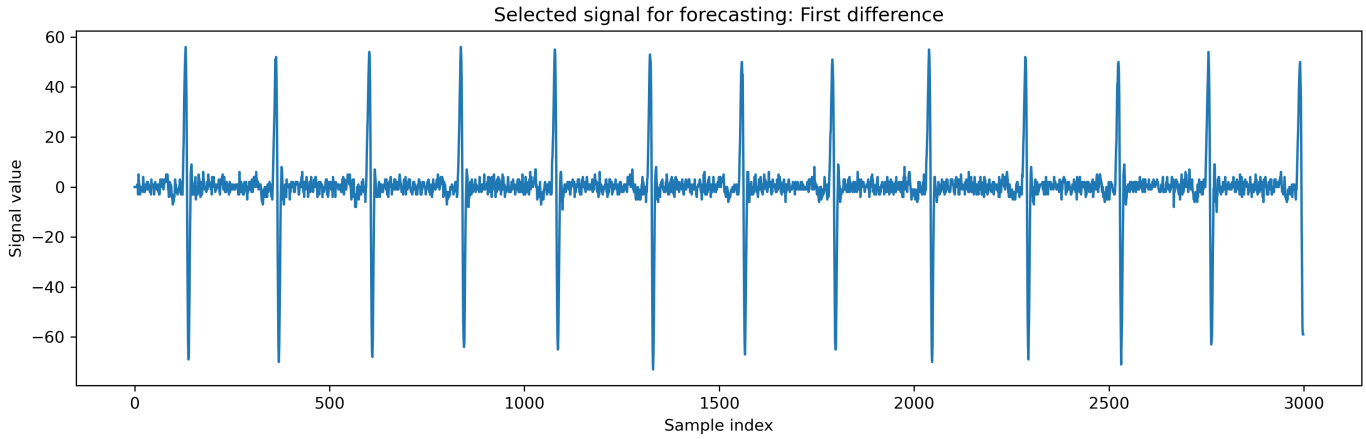

Figure 8. First-difference ECG signal selected for the forecasting stage.

Figure 9 shows the seasonal decomposition of the selected signal.

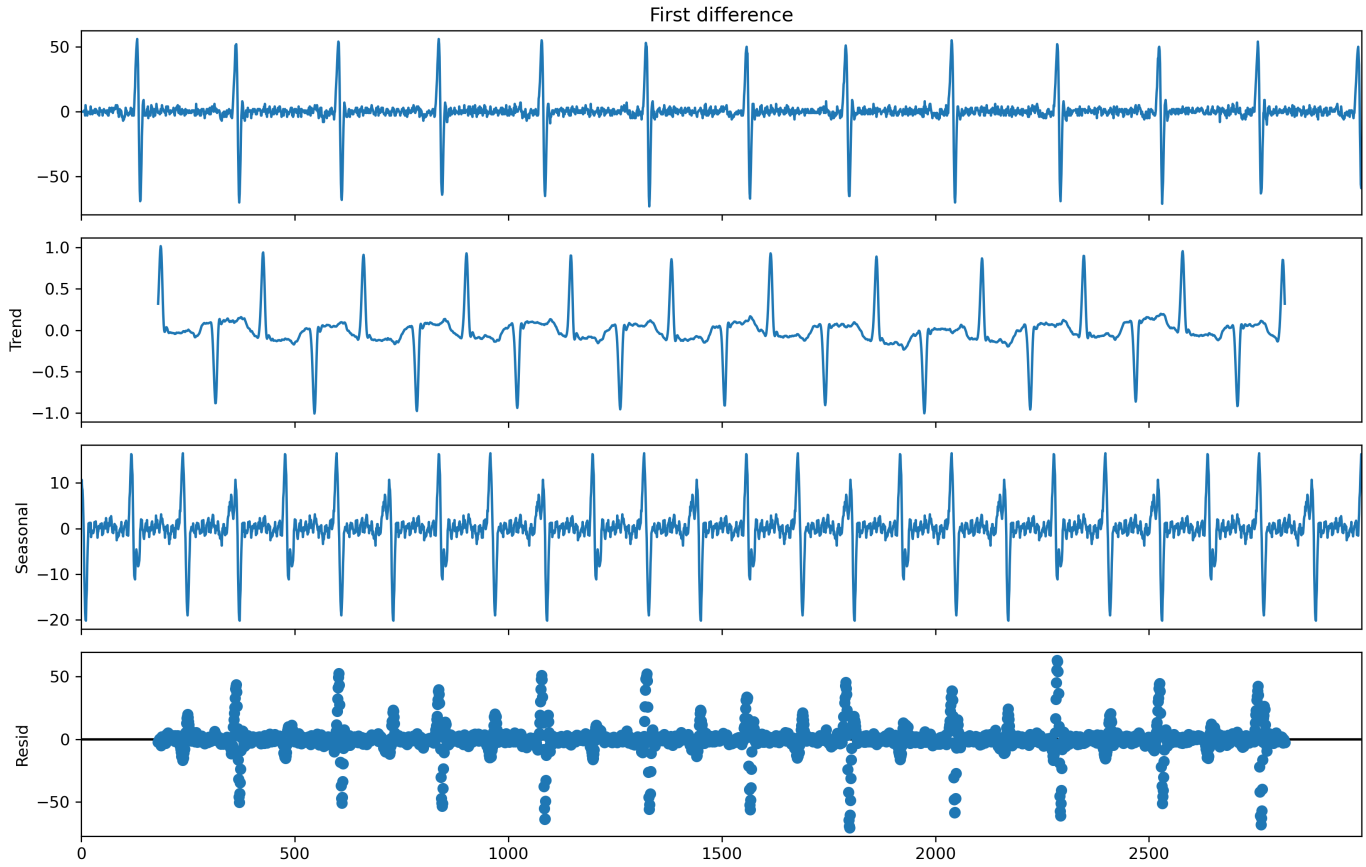

Figure 9. Seasonal decomposition of the selected first-difference ECG signal using period 360.

Figure 10 shows the ACF and PACF plots used to select the AR and MA orders.

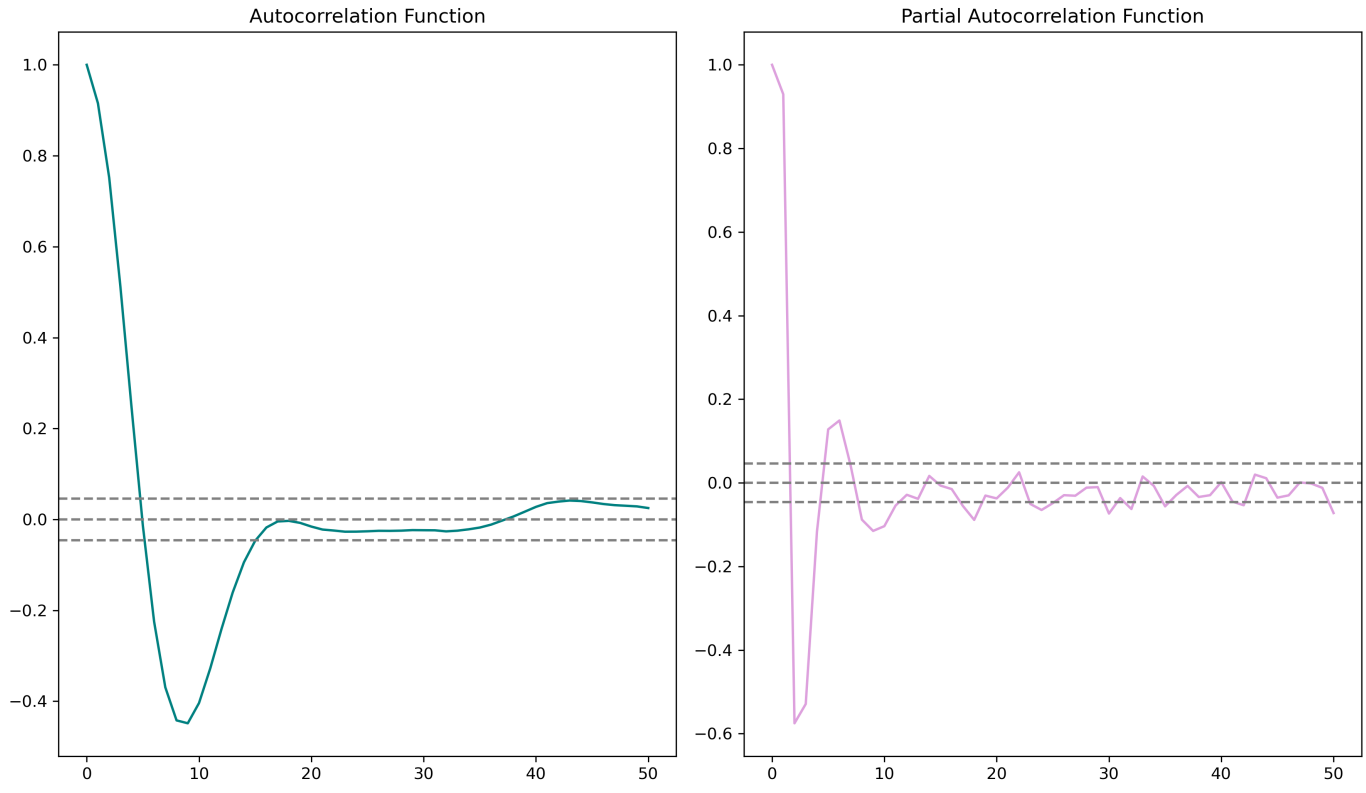

Figure 10. ACF and PACF plots of the selected training signal, with significance bounds.

Figure 11 compares the forecasts from the AR, MA, ARMA, and ARIMA models against the test signal.

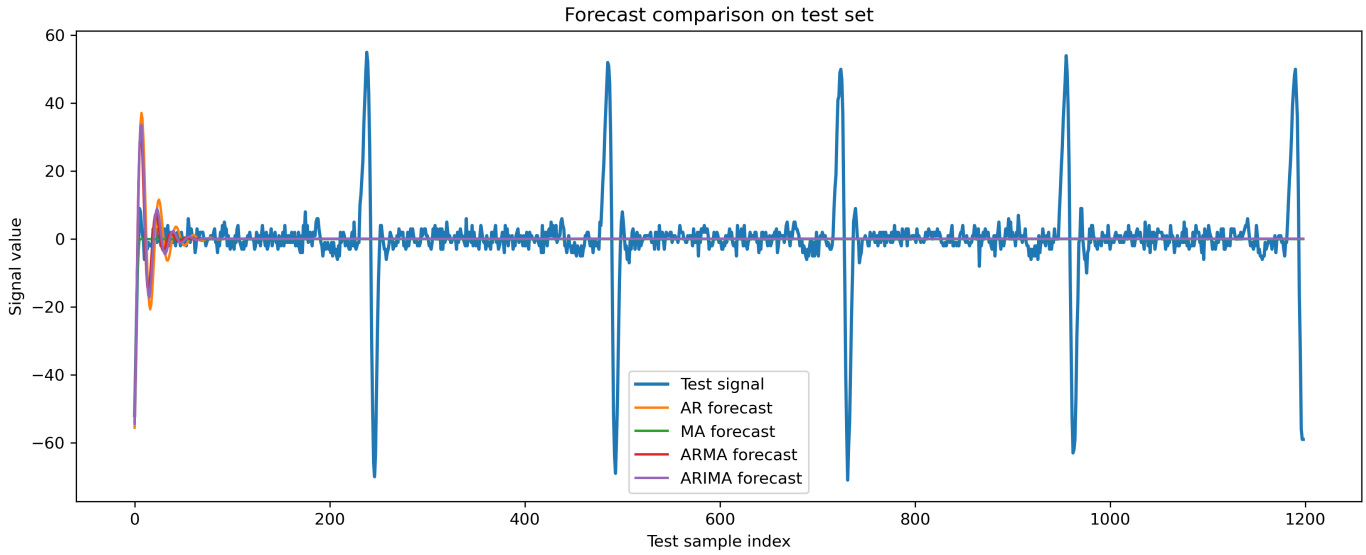

Figure 11. Forecast comparison for AR, MA, ARMA, and ARIMA models on the test set.

The first-difference signal was selected as it achieved the lowest p-value, indicating the strongest stationarity.

Question 5

The Statlog heart dataset was binarised using the thresholds given in the question. Each binary value was then one-hot encoded as an association-analysis item, including `class=present` and `class=absent`. Following Tutorial 9, Apriori was used to find frequent itemsets, and association rules were extracted using lift.

The Apriori support threshold was 0.25, the lift threshold was 1.15, and the final significant rules were selected using conviction greater than 1.5.

Output	Count
Patient records	270
Frequent itemsets, support >= 0.25	572
Association rules, lift >= 1.15	1694
Significant rules, conviction > 1.5	841

Most frequent itemsets:

Itemset	Support
`fasting_blood_sugar=0`	0.8519
`chest>2.5`	0.7704
`maximum_heart_rate_achieved>140`	0.6963
`oldpeak!=0`	0.6852
`age>50`	0.6815
`sex=1`	0.6778

Disease-related rules with high conviction:

Antecedents	Consequent	Support	Confidence	Lift	Conviction
`exercise_induced_angina=0`, `maximum_heart_rate_achieved>140`, `number_of_major_vessels=0`, `thal=3`	`class=absent`	0.2630	0.9103	1.6385	4.9524
`maximum_heart_rate_achieved>140`, `number_of_major_vessels=0`, `thal=3`	`class=absent`	0.3074	0.9022	1.6239	4.5432
`exercise_induced_angina=0`, `number_of_major_vessels=0`, `thal=3`	`class=absent`	0.3037	0.9011	1.6220	4.4938
`number_of_major_vessels!=0`, `sex=1`	`class=present`	0.2519	0.8293	1.8659	3.2540
`chest>2.5`, `oldpeak!=0`, `thal!=3`	`class=present`	0.2667	0.8090	1.8202	2.9085
`chest>2.5`, `thal!=3`	`class=present`	0.2963	0.7921	1.7822	2.6720

The strongest disease-related rules were more indicative of disease absence. These rules combined no exercise-induced angina, high maximum heart rate, no major vessels, and normal thal. Disease presence was mainly associated with non-zero major vessels, high chest-pain category, non-zero oldpeak, and abnormal thal.

These rules are consistent with expected clinical patterns, where normal conditions are associated with disease absence and abnormal indicators with disease presence.

Appendix

q1

```
import os
import itertools
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import (
    confusion_matrix,
```

```
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)


CSV_PATH = "ecg_signals_preprocessed.csv"
OUTPUT_DIR = "outputs_q1"

RANDOM_STATE = 4
N_CLUSTERS = 3
CLASS_NAMES = ["N", "S", "V"]

os.makedirs(OUTPUT_DIR, exist_ok=True)


def load_data(csv_path):
    df = pd.read_csv(csv_path)

    X = df.iloc[:, :-1].values
    y_true = df.iloc[:, -1].values.astype(int)

    return X, y_true, df


def map_cluster_labels_to_true_labels(y_true, y_cluster):
    # Cluster labels have no class meaning, so test all label mappings.
    labels = np.arange(N_CLUSTERS)
    best_accuracy = -1
    best_mapping = None
    best_y_mapped = None

    for permutation in itertools.permutations(labels):
        cluster_to_label = {cluster_id: class_id for cluster_id, class_id in zip(labels,
permutation)}
        y_mapped = np.array([cluster_to_label[c] for c in y_cluster])
        current_accuracy = accuracy_score(y_true, y_mapped)

        if current_accuracy > best_accuracy:
            best_accuracy = current_accuracy
            best_mapping = cluster_to_label
            best_y_mapped = y_mapped

    return best_y_mapped, best_mapping


def evaluate_clustering(y_true, y_cluster, experiment_name):
    y_pred, mapping = map_cluster_labels_to_true_labels(y_true, y_cluster)

    cm = confusion_matrix(y_true, y_pred)
```

```
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, average="macro", zero_division=0)
    recall = recall_score(y_true, y_pred, average="macro", zero_division=0)
    f1 = f1_score(y_true, y_pred, average="macro", zero_division=0)

    print("\n" + "=" * 70)
    print(experiment_name)
    print("=" * 70)
    print("Cluster-to-class mapping:")
    print(mapping)
    print("\nConfusion matrix:")
    print(cm)
    print(f"\nAccuracy:        {accuracy:.4f}")
    print(f"Macro Precision: {precision:.4f}")
    print(f"Macro Recall:    {recall:.4f}")
    print(f"Macro F1-score:  {f1:.4f}")

    print("\nClassification report:")
    print(
        classification_report(
            y_true,
            y_pred,
            target_names=CLASS_NAMES,
            zero_division=0
        )
    )

    results = {
        "experiment": experiment_name,
        "mapping": mapping,
        "confusion_matrix": cm,
        "accuracy": accuracy,
        "precision_macro": precision,
        "recall_macro": recall,
        "f1_macro": f1
    }

    return results, y_pred


def plot_confusion_matrix(cm, title, save_path):
    plt.figure(figsize=(5.5, 4.5))
    plt.imshow(cm)
    plt.title(title)
    plt.xlabel("Predicted class")
    plt.ylabel("True class")

    plt.xticks(np.arange(len(CLASS_NAMES)), CLASS_NAMES)
    plt.yticks(np.arange(len(CLASS_NAMES)), CLASS_NAMES)

    for i in range(cm.shape[0]):
```

```
        for j in range(cm.shape[1]):
            plt.text(
                j,
                i,
                str(cm[i, j]),
                ha="center",
                va="center"
            )

    plt.colorbar(label="Number of samples")
    plt.tight_layout()
    plt.savefig(save_path, dpi=300)
    plt.close()


def save_results_table(results_list, save_path):
    rows = []

    for r in results_list:
        rows.append({
            "Experiment": r["experiment"],
            "Accuracy": r["accuracy"],
            "Macro Precision": r["precision_macro"],
            "Macro Recall": r["recall_macro"],
            "Macro F1-score": r["f1_macro"]
        })

    table = pd.DataFrame(rows)
    table.to_csv(save_path, index=False)

    print("\nSummary table:")
    print(table)

    return table


def main():
    X, y_true, df = load_data(CSV_PATH)

    print("Dataset loaded.")
    print(f"Feature matrix shape: {X.shape}")
    print(f"Label vector shape:   {y_true.shape}")
    print(f"Class distribution:   {np.bincount(y_true)}")

    x = X
    min_max_scaler = MinMaxScaler()
    X_scaled = min_max_scaler.fit_transform(x)

    # Cluster with all 83 features.
    kmeans_all = KMeans(
        n_clusters=N_CLUSTERS,
        random_state=RANDOM_STATE
```

```
)

cluster_labels_all = kmeans_all.fit(X_scaled).predict(X_scaled)

results_all, y_pred_all = evaluate_clustering(
    y_true,
    cluster_labels_all,
    "KMeans using all 83 normalised features"
)

plot_confusion_matrix(
    results_all["confusion_matrix"],
    "KMeans Confusion Matrix - All 83 Features",
    os.path.join(OUTPUT_DIR, "q1_kmeans_all_features_confusion.png")
)

# Keep enough PCA components to explain at least 90% of variance.
pca_full = PCA()
pca_full.fit(X_scaled)

cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
n_components_90 = np.argmax(cumulative_variance >= 0.90) + 1

print("\n" + "=" * 70)
print("PCA dimensionality reduction")
print("=" * 70)
print(f"Number of components needed for >90% variance: {n_components_90}")
print(
    f"Cumulative explained variance retained: "
    f"{cumulative_variance[n_components_90 - 1]:.4f}"
)

pca = PCA(n_components=0.90, whiten=True)
X_pca = pca.fit_transform(X_scaled)
n_components_90 = X_pca.shape[1]

# Save the cumulative variance plot for PCA.
plt.figure(figsize=(7, 4))
plt.plot(
    np.arange(1, len(cumulative_variance) + 1),
    cumulative_variance,
    marker="o"
)
plt.axhline(y=0.90, linestyle="--", label="90% variance threshold")
plt.axvline(x=n_components_90, linestyle="--", label=f"{n_components_90} components")
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative explained variance")
plt.title("PCA cumulative explained variance")
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "q1_pca_cumulative_variance.png"), dpi=300)
plt.close()
```

```
    # Cluster with PCA-transformed features.
    kmeans_pca = KMeans(
        n_clusters=N_CLUSTERS,
        random_state=RANDOM_STATE
    )

    cluster_labels_pca = kmeans_pca.fit(X_pca).predict(X_pca)

    results_pca, y_pred_pca = evaluate_clustering(
        y_true,
        cluster_labels_pca,
        "KMeans after PCA with >90% explained variance"
    )

    plot_confusion_matrix(
        results_pca["confusion_matrix"],
        "KMeans Confusion Matrix - PCA Features",
        os.path.join(OUTPUT_DIR, "q1_kmeans_pca_confusion.png")
    )

    # Save the result table.
    save_results_table(
        [results_all, results_pca],
        os.path.join(OUTPUT_DIR, "q1_kmeans_summary.csv")
    )

    print("\nGenerated output files:")
    print(f"- {OUTPUT_DIR}/q1_kmeans_all_features_confusion.png")
    print(f"- {OUTPUT_DIR}/q1_pca_cumulative_variance.png")
    print(f"- {OUTPUT_DIR}/q1_kmeans_pca_confusion.png")
    print(f"- {OUTPUT_DIR}/q1_kmeans_summary.csv")


if __name__ == "__main__":
    main()
```

q2

```
import os
import itertools
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
```

```
    precision_score,
    recall_score,
    f1_score,
    classification_report
)


CSV_PATH = "ecg_signals_preprocessed.csv"
OUTPUT_DIR = "outputs_q2"

RANDOM_STATE = 9
N_COMPONENTS_GMM = 3
CLASS_NAMES = ["N", "S", "V"]

os.makedirs(OUTPUT_DIR, exist_ok=True)


def load_data(csv_path):
    df = pd.read_csv(csv_path)

    X = df.iloc[:, :-1].values
    y_true = df.iloc[:, -1].values.astype(int)

    return X, y_true, df


def map_cluster_labels_to_true_labels(y_true, y_cluster):
    labels = np.arange(N_COMPONENTS_GMM)
    best_accuracy = -1
    best_mapping = None
    best_y_mapped = None

    for permutation in itertools.permutations(labels):
        cluster_to_label = {cluster_id: class_id for cluster_id, class_id in zip(labels,
permutation)}
        y_mapped = np.array([cluster_to_label[c] for c in y_cluster])
        current_accuracy = accuracy_score(y_true, y_mapped)

        if current_accuracy > best_accuracy:
            best_accuracy = current_accuracy
            best_mapping = cluster_to_label
            best_y_mapped = y_mapped

    return best_y_mapped, best_mapping


def evaluate_clustering(y_true, y_cluster, experiment_name):
    y_pred, mapping = map_cluster_labels_to_true_labels(y_true, y_cluster)

    cm = confusion_matrix(y_true, y_pred)

    accuracy = accuracy_score(y_true, y_pred)
```

```
    precision = precision_score(y_true, y_pred, average="macro", zero_division=0)
    recall = recall_score(y_true, y_pred, average="macro", zero_division=0)
    f1 = f1_score(y_true, y_pred, average="macro", zero_division=0)

    print("\n" + "=" * 70)
    print(experiment_name)
    print("=" * 70)
    print("Cluster-to-class mapping:")
    print(mapping)
    print("\nConfusion matrix:")
    print(cm)
    print(f"\nAccuracy:        {accuracy:.4f}")
    print(f"Macro Precision: {precision:.4f}")
    print(f"Macro Recall:    {recall:.4f}")
    print(f"Macro F1-score:  {f1:.4f}")

    print("\nClassification report:")
    print(
        classification_report(
            y_true,
            y_pred,
            target_names=CLASS_NAMES,
            zero_division=0
        )
    )

    results = {
        "experiment": experiment_name,
        "mapping": mapping,
        "confusion_matrix": cm,
        "accuracy": accuracy,
        "precision_macro": precision,
        "recall_macro": recall,
        "f1_macro": f1
    }

    return results, y_pred


def plot_confusion_matrix(cm, title, save_path):
    plt.figure(figsize=(5.5, 4.5))
    plt.imshow(cm)
    plt.title(title)
    plt.xlabel("Predicted class")
    plt.ylabel("True class")

    plt.xticks(np.arange(len(CLASS_NAMES)), CLASS_NAMES)
    plt.yticks(np.arange(len(CLASS_NAMES)), CLASS_NAMES)

    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, str(cm[i, j]), ha="center", va="center")
```

```
    plt.colorbar(label="Number of samples")
    plt.tight_layout()
    plt.savefig(save_path, dpi=300)
    plt.close()


def save_results_table(results_list, save_path):
    rows = []

    for r in results_list:
        rows.append({
            "Experiment": r["experiment"],
            "Accuracy": r["accuracy"],
            "Macro Precision": r["precision_macro"],
            "Macro Recall": r["recall_macro"],
            "Macro F1-score": r["f1_macro"]
        })

    table = pd.DataFrame(rows)
    table.to_csv(save_path, index=False)

    print("\nSummary table:")
    print(table)

    return table


def main():
    X, y_true, df = load_data(CSV_PATH)
    x = X
    min_max_scaler = MinMaxScaler()
    X_scaled = min_max_scaler.fit_transform(x)

    # Train GMM-EM with all 83 features.
    gmm_all = GaussianMixture(
        n_components=N_COMPONENTS_GMM,
        covariance_type="full",
        random_state=RANDOM_STATE
    )

    gmm_all.fit(X_scaled)
    cluster_labels_all = gmm_all.predict(X_scaled)

    results_all, y_pred_all = evaluate_clustering(
        y_true,
        cluster_labels_all,
        "GMM-EM using all 83 normalised features"
    )

    plot_confusion_matrix(
        results_all["confusion_matrix"],
```

```
    "GMM-EM Confusion Matrix - All 83 Features",
    os.path.join(OUTPUT_DIR, "q2_gmm_all_features_confusion.png")
)

# Keep enough PCA components to explain at least 90% of variance.
pca_full = PCA()
pca_full.fit(X_scaled)

cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
n_components_90 = np.argmax(cumulative_variance >= 0.90) + 1
retained_variance = cumulative_variance[n_components_90 - 1]

pca = PCA(n_components=0.90, whiten=True)
X_pca = pca.fit_transform(X_scaled)
n_components_90 = X_pca.shape[1]
retained_variance = np.sum(pca.explained_variance_ratio_)

# Save the cumulative variance plot for PCA.
plt.figure(figsize=(7, 4))
plt.plot(
    np.arange(1, len(cumulative_variance) + 1),
    cumulative_variance,
    marker="o"
)
plt.axhline(y=0.90, linestyle="--", label="90% variance threshold")
plt.axvline(
    x=n_components_90,
    linestyle="--",
    label=f"{n_components_90} components"
)
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative explained variance")
plt.title("PCA cumulative explained variance")
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "q2_pca_cumulative_variance.png"), dpi=300)
plt.close()

# Train GMM-EM with PCA-transformed features.
gmm_pca = GaussianMixture(
    n_components=N_COMPONENTS_GMM,
    covariance_type="full",
    random_state=RANDOM_STATE
)

gmm_pca.fit(X_pca)
cluster_labels_pca = gmm_pca.predict(X_pca)

results_pca, y_pred_pca = evaluate_clustering(
    y_true,
    cluster_labels_pca,
    "GMM-EM after PCA with >90% explained variance"
```

```
    )

    plot_confusion_matrix(
        results_pca["confusion_matrix"],
        "GMM-EM Confusion Matrix - PCA Features",
        os.path.join(OUTPUT_DIR, "q2_gmm_pca_confusion.png")
    )

    # Save the result table.
    save_results_table(
        [results_all, results_pca],
        os.path.join(OUTPUT_DIR, "q2_gmm_summary.csv")
    )


if __name__ == "__main__":
    main()
```

q3

```
import os
import itertools
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)


CSV_PATH = "ecg_signals_preprocessed.csv"
OUTPUT_DIR = "outputs_q3"

N_CLUSTERS = 3
CLASS_NAMES = ["N", "S", "V"]

# Test two linkage methods.
LINKAGE_METHODS = ["average", "complete"]

os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```
def load_data(csv_path):
    df = pd.read_csv(csv_path)

    X = df.iloc[:, :-1].values
    y_true = df.iloc[:, -1].values.astype(int)

    return X, y_true, df


def map_cluster_labels_to_true_labels(y_true, y_cluster):
    # Cluster labels have no class meaning, so test all label mappings.
    labels = np.arange(N_CLUSTERS)
    best_accuracy = -1
    best_mapping = None
    best_y_mapped = None

    for permutation in itertools.permutations(labels):
        cluster_to_label = {cluster_id: class_id for cluster_id, class_id in zip(labels,
permutation)}
        y_mapped = np.array([cluster_to_label[c] for c in y_cluster])
        current_accuracy = accuracy_score(y_true, y_mapped)

        if current_accuracy > best_accuracy:
            best_accuracy = current_accuracy
            best_mapping = cluster_to_label
            best_y_mapped = y_mapped

    return best_y_mapped, best_mapping


def evaluate_clustering(y_true, y_cluster, experiment_name):
    y_pred, mapping = map_cluster_labels_to_true_labels(y_true, y_cluster)

    cm = confusion_matrix(y_true, y_pred)

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, average="macro", zero_division=0)
    recall = recall_score(y_true, y_pred, average="macro", zero_division=0)
    f1 = f1_score(y_true, y_pred, average="macro", zero_division=0)

    print("\n" + "=" * 80)
    print(experiment_name)
    print("=" * 80)
    print("Cluster-to-class mapping:")
    print(mapping)
    print("\nConfusion matrix:")
    print(cm)
    print(f"\nAccuracy:        {accuracy:.4f}")
    print(f"Macro Precision: {precision:.4f}")
    print(f"Macro Recall:    {recall:.4f}")
    print(f"Macro F1-score:  {f1:.4f}")
```

```
    print("\nClassification report:")
    print(
        classification_report(
            y_true,
            y_pred,
            target_names=CLASS_NAMES,
            zero_division=0
        )
    )

    results = {
        "experiment": experiment_name,
        "mapping": mapping,
        "confusion_matrix": cm,
        "accuracy": accuracy,
        "precision_macro": precision,
        "recall_macro": recall,
        "f1_macro": f1
    }

    return results, y_pred


def plot_confusion_matrix(cm, title, save_path):
    plt.figure(figsize=(5.5, 4.5))
    plt.imshow(cm)
    plt.title(title)
    plt.xlabel("Predicted class")
    plt.ylabel("True class")

    plt.xticks(np.arange(len(CLASS_NAMES)), CLASS_NAMES)
    plt.yticks(np.arange(len(CLASS_NAMES)), CLASS_NAMES)

    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, str(cm[i, j]), ha="center", va="center")

    plt.colorbar(label="Number of samples")
    plt.tight_layout()
    plt.savefig(save_path, dpi=300)
    plt.close()


def results_to_dataframe(results_dict):
    rows = []

    for linkage, r in results_dict.items():
        rows.append({
            "Linkage": linkage,
            "Experiment": r["experiment"],
            "Accuracy": r["accuracy"],
            "Macro Precision": r["precision_macro"],
```

```
            "Macro Recall": r["recall_macro"],
            "Macro F1-score": r["f1_macro"]
        })

    df = pd.DataFrame(rows)
    df = df.sort_values(by="Accuracy", ascending=False)

    return df


def run_agglomerative_experiments(X_features, y_true, feature_type_name):
    all_results = {}

    for linkage in LINKAGE_METHODS:
        agglom = AgglomerativeClustering(
            n_clusters=N_CLUSTERS,
            linkage=linkage
        )

        agglom.fit(X_features)
        cluster_labels = agglom.labels_

        results, y_pred = evaluate_clustering(
            y_true,
            cluster_labels,
            f"Agglomerative clustering using {feature_type_name}, linkage={linkage}"
        )

        all_results[linkage] = results

    return all_results


def main():
    X, y_true, df = load_data(CSV_PATH)
    x = X
    min_max_scaler = MinMaxScaler()
    X_scaled = min_max_scaler.fit_transform(x)
    results_all = run_agglomerative_experiments(
        X_scaled,
        y_true,
        "all 83 normalised features"
    )

    summary_all = results_to_dataframe(results_all)
    summary_all_path = os.path.join(
        OUTPUT_DIR,
        "q3_agglomerative_all_linkage_summary.csv"
    )
    summary_all.to_csv(summary_all_path, index=False)

    print("\nSummary: Agglomerative clustering using all 83 features")
```

```
print(summary_all)

best_linkage_all = summary_all.iloc[0]["Linkage"]
best_results_all = results_all[best_linkage_all]

plot_confusion_matrix(
    best_results_all["confusion_matrix"],
    f"Agglomerative Confusion Matrix - All Features - {best_linkage_all}",
    os.path.join(
        OUTPUT_DIR,
        "q3_agglomerative_best_all_features_confusion.png"
    )
)
pca_full = PCA()
pca_full.fit(X_scaled)

cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
n_components_90 = np.argmax(cumulative_variance >= 0.90) + 1
retained_variance = cumulative_variance[n_components_90 - 1]

pca = PCA(n_components=0.90, whiten=True)
X_pca = pca.fit_transform(X_scaled)
n_components_90 = X_pca.shape[1]
retained_variance = np.sum(pca.explained_variance_ratio_)

plt.figure(figsize=(7, 4))
plt.plot(
    np.arange(1, len(cumulative_variance) + 1),
    cumulative_variance,
    marker="o"
)
plt.axhline(y=0.90, linestyle="--", label="90% variance threshold")
plt.axvline(
    x=n_components_90,
    linestyle="--",
    label=f"{n_components_90} components"
)
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative explained variance")
plt.title("PCA cumulative explained variance")
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, "q3_pca_cumulative_variance.png"), dpi=300)
plt.close()

# Cluster with PCA-transformed features.
results_pca = run_agglomerative_experiments(
    X_pca,
    y_true,
    f"PCA features ({n_components_90} components, {retained_variance:.4f} variance)"
)
```

```
summary_pca = results_to_dataframe(results_pca)
summary_pca_path = os.path.join(
    OUTPUT_DIR,
    "q3_agglomerative_pca_linkage_summary.csv"
)
summary_pca.to_csv(summary_pca_path, index=False)

print("\nSummary: Agglomerative clustering after PCA")
print(summary_pca)

best_linkage_pca = summary_pca.iloc[0]["Linkage"]
best_results_pca = results_pca[best_linkage_pca]

plot_confusion_matrix(
    best_results_pca["confusion_matrix"],
    f"Agglomerative Confusion Matrix - PCA - {best_linkage_pca}",
    os.path.join(
        OUTPUT_DIR,
        "q3_agglomerative_best_pca_confusion.png"
    )
)

# Save the final comparison table.
final_comparison = pd.DataFrame([
    {
        "Feature representation": "All 83 normalised features",
        "Best linkage": best_linkage_all,
        "Accuracy": best_results_all["accuracy"],
        "Macro Precision": best_results_all["precision_macro"],
        "Macro Recall": best_results_all["recall_macro"],
        "Macro F1-score": best_results_all["f1_macro"]
    },
    {
        "Feature representation": f"PCA features ({n_components_90} components)",
        "Best linkage": best_linkage_pca,
        "Accuracy": best_results_pca["accuracy"],
        "Macro Precision": best_results_pca["precision_macro"],
        "Macro Recall": best_results_pca["recall_macro"],
        "Macro F1-score": best_results_pca["f1_macro"]
    }
])

final_comparison_path = os.path.join(
    OUTPUT_DIR,
    "q3_agglomerative_final_comparison.csv"
)
final_comparison.to_csv(final_comparison_path, index=False)

print("\nFinal Q3 comparison:")
print(final_comparison)

# Print class-level recall to support the confusion matrix discussion.
```

```
    cm_best = best_results_all["confusion_matrix"]
    class_recalls = cm_best.diagonal() / cm_best.sum(axis=1)

    print("\nClass-level recall based on best all-feature result:")
    for class_name, class_recall in zip(CLASS_NAMES, class_recalls):
        print(f"Recall for class {class_name}: {class_recall:.4f}")

    easiest_class = CLASS_NAMES[np.argmax(class_recalls)]
    hardest_class = CLASS_NAMES[np.argmin(class_recalls)]

    print(f"Easiest class to cluster according to recall: {easiest_class}")
    print(f"Most difficult class to cluster according to recall: {hardest_class}")


if __name__ == "__main__":
    main()
```

q4

```
import os
import warnings

warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller, acf, pacf
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA


CSV_PATH = "single_ecg_signal.csv"
OUTPUT_DIR = "outputs_q4"

TRAIN_SIZE = 1800
SAMPLING_RATE = 360
MAX_MODEL_ORDER = 5

os.makedirs(OUTPUT_DIR, exist_ok=True)


def load_signal(csv_path):
    df = pd.read_csv(csv_path)

    signal = df.iloc[:, 0].astype(float).values

    return signal
```

```python
def run_adf_test(series, name):
    clean_series = pd.Series(series).replace([np.inf, -np.inf], np.nan).dropna()

    result = adfuller(clean_series)

    adf_statistic = result[0]
    p_value = result[1]
    used_lags = result[2]
    n_observations = result[3]
    critical_values = result[4]

    return {
        "Signal version": name,
        "ADF statistic": adf_statistic,
        "p-value": p_value,
        "Used lags": used_lags,
        "No. observations": n_observations,
        "Critical value 1%": critical_values["1%"],
        "Critical value 5%": critical_values["5%"],
        "Critical value 10%": critical_values["10%"],
        "Stationary at 5%": p_value < 0.05
    }


def create_transformations(signal):
    original = pd.Series(signal, name="Original")

    first_difference = original.diff().dropna()
    first_difference.name = "First difference"

    min_value = original.min()
    shifted = original - min_value + 1e-6

    square_root = np.sqrt(shifted)
    square_root = pd.Series(square_root, name="Square root")

    log_transform = np.log(shifted)
    log_transform = pd.Series(log_transform, name="Log")

    transformations = {
        "Original": original,
        "First difference": first_difference,
        "Square root": square_root,
        "Log": log_transform
    }

    return transformations


def stationarity_analysis(transformations):
    adf_rows = []
```

```
    for name, series in transformations.items():
        adf_rows.append(run_adf_test(series, name))

    adf_df = pd.DataFrame(adf_rows)

    adf_df_sorted = adf_df.sort_values(
        by=["p-value", "ADF statistic"],
        ascending=[True, True]
    )

    selected_name = adf_df_sorted.iloc[0]["Signal version"]
    selected_signal = transformations[selected_name].dropna().reset_index(drop=True)

    adf_path = os.path.join(OUTPUT_DIR, "q4_adf_results.csv")
    adf_df.to_csv(adf_path, index=False)

    print("\nADF test results:")
    print(adf_df)

    return adf_df, selected_name, selected_signal


def plot_original_signal(signal):
    plt.figure(figsize=(12, 4))
    plt.plot(signal)
    plt.title("Original ECG signal")
    plt.xlabel("Sample index")
    plt.ylabel("Amplitude")
    plt.tight_layout()
    plt.savefig(os.path.join(OUTPUT_DIR, "q4_original_signal.png"), dpi=300)
    plt.close()


def plot_transformed_signals(transformations):
    for name, series in transformations.items():
        safe_name = name.lower().replace(" ", "_")

        plt.figure(figsize=(12, 4))
        plt.plot(series.values)
        plt.title(f"ECG signal - {name}")
        plt.xlabel("Sample index")
        plt.ylabel("Transformed amplitude")
        plt.tight_layout()
        plt.savefig(
            os.path.join(OUTPUT_DIR, f"q4_signal_{safe_name}.png"),
            dpi=300
        )
        plt.close()


def plot_selected_signal(selected_signal, selected_name):
```

```
    plt.figure(figsize=(12, 4))
    plt.plot(selected_signal.values)
    plt.title(f"Selected signal for forecasting: {selected_name}")
    plt.xlabel("Sample index")
    plt.ylabel("Signal value")
    plt.tight_layout()
    plt.savefig(os.path.join(OUTPUT_DIR, "q4_selected_signal.png"), dpi=300)
    plt.close()


def estimate_seasonal_period(signal, sampling_rate=360):
    return sampling_rate


def perform_seasonal_decomposition(selected_signal, selected_name):
    period = estimate_seasonal_period(selected_signal)

    try:
        decomposition = seasonal_decompose(
            selected_signal,
            model="additive",
            period=period
        )

        fig = decomposition.plot()
        fig.set_size_inches(12, 8)
        fig.suptitle(
            f"Seasonal decomposition of selected signal: {selected_name}",
            y=1.02
        )
        plt.tight_layout()
        plt.savefig(
            os.path.join(OUTPUT_DIR, "q4_seasonal_decomposition.png"),
            dpi=300
        )
        plt.close()

        return period

    except Exception as e:
        print(f"Seasonal decomposition failed: {e}")
        return period


def suggest_orders_from_acf_pacf(train, max_lag=40, max_order=5):
    train_values = pd.Series(train).dropna().values
    n = len(train_values)

    significance_threshold = 1.96 / np.sqrt(n)

    acf_values = acf(train_values, nlags=max_lag, fft=True)
    pacf_values = pacf(train_values, nlags=max_lag, method="ols")
```

```
    # Exclude lag 0.
    significant_acf_lags = np.where(np.abs(acf_values[1:]) > significance_threshold)[0] +
1
    significant_pacf_lags = np.where(np.abs(pacf_values[1:]) > significance_threshold)[0]
+ 1

    if len(significant_pacf_lags) == 0:
        p = 1
    else:
        p = int(min(significant_pacf_lags[-1], max_order))

    if len(significant_acf_lags) == 0:
        q = 1
    else:
        q = int(min(significant_acf_lags[-1], max_order))

    p = max(1, p)
    q = max(1, q)

    return p, q, significance_threshold


def plot_acf_pacf_figures(train):
    y = pd.Series(train).dropna().values
    lag_acf = acf(y, nlags=50)
    lag_pacf = pacf(y, nlags=50, method="ols")

    fig = plt.figure(figsize=(12, 7))

    plt.subplot(121)
    plt.plot(lag_acf, color="teal")
    plt.axhline(y=0, linestyle="--", color="gray")
    plt.axhline(y=-1.96 / np.sqrt(len(y)), linestyle="--", color="gray")
    plt.axhline(y=1.96 / np.sqrt(len(y)), linestyle="--", color="gray")
    plt.title("Autocorrelation Function")

    plt.subplot(122)
    plt.plot(lag_pacf, color="plum")
    plt.axhline(y=0, linestyle="--", color="gray")
    plt.axhline(y=-1.96 / np.sqrt(len(y)), linestyle="--", color="gray")
    plt.axhline(y=1.96 / np.sqrt(len(y)), linestyle="--", color="gray")
    plt.title("Partial Autocorrelation Function")

    plt.tight_layout()
    plt.savefig(os.path.join(OUTPUT_DIR, "q4_acf_pacf.png"), dpi=300)
    plt.close()


def calculate_rss(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
```

```
    return np.sum((y_true - y_pred) ** 2)


def fit_forecast_model(train, test, order, model_name):
    model = ARIMA(train, order=order)
    fitted_model = model.fit()

    forecast = fitted_model.forecast(steps=len(test))
    model_rss = calculate_rss(test, forecast)

    return {
        "Model": model_name,
        "Order": order,
        "RSS": model_rss,
        "Forecast": forecast,
        "AIC": fitted_model.aic,
        "BIC": fitted_model.bic
    }


def run_forecasting_models(train, test, p, q):
    model_specs = [
        ("AR", (p, 0, 0)),
        ("MA", (0, 0, q)),
        ("ARMA", (p, 0, q)),
        ("ARIMA", (p, 1, q))
    ]

    results = []

    for model_name, order in model_specs:
        try:
            result = fit_forecast_model(train, test, order, model_name)
            results.append(result)
        except Exception as e:
            print(f"{model_name} with order={order} failed: {e}")

            # Use a simpler order if fitting fails.
            fallback_order = {
                "AR": (1, 0, 0),
                "MA": (0, 0, 1),
                "ARMA": (1, 0, 1),
                "ARIMA": (1, 1, 1)
            }[model_name]

            try:
                result = fit_forecast_model(train, test, fallback_order, model_name)
                results.append(result)
            except Exception as e2:
                print(f"Fallback {model_name} also failed: {e2}")
```

```
    return results


def save_model_comparison(results):
    rows = []

    for r in results:
        rows.append({
            "Model": r["Model"],
            "Order": r["Order"],
            "RSS": r["RSS"],
            "AIC": r["AIC"],
            "BIC": r["BIC"]
        })

    comparison_df = pd.DataFrame(rows)
    comparison_df = comparison_df.sort_values(by="RSS", ascending=True)

    output_path = os.path.join(OUTPUT_DIR, "q4_model_comparison.csv")
    comparison_df.to_csv(output_path, index=False)

    print("\nModel comparison table:")
    print(comparison_df)

    return comparison_df


def plot_forecast_comparison(test, results):
    plt.figure(figsize=(12, 5))

    plt.plot(
        np.arange(len(test)),
        test.values,
        label="Test signal",
        linewidth=2
    )

    for r in results:
        forecast = np.asarray(r["Forecast"])
        plt.plot(
            np.arange(len(forecast)),
            forecast,
            label=f"{r['Model']} forecast"
        )

    plt.title("Forecast comparison on test set")
    plt.xlabel("Test sample index")
    plt.ylabel("Signal value")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(OUTPUT_DIR, "q4_forecast_comparison.png"), dpi=300)
    plt.close()
```

```python
def main():
    # Load the signal.
    signal = load_signal(CSV_PATH)

    plot_original_signal(signal)

    # Build transformed signals.
    transformations = create_transformations(signal)

    plot_transformed_signals(transformations)

    # Run ADF stationarity tests.
    adf_df, selected_name, selected_signal = stationarity_analysis(transformations)

    plot_selected_signal(selected_signal, selected_name)

    # Split the train and test sets.
    if len(selected_signal) <= TRAIN_SIZE:
        raise ValueError(
            f"Selected signal length is {len(selected_signal)}, "
            f"which is not greater than TRAIN_SIZE={TRAIN_SIZE}."
        )

    train = selected_signal.iloc[:TRAIN_SIZE]
    test = selected_signal.iloc[TRAIN_SIZE:]

    # Seasonal decomposition.
    seasonal_period = perform_seasonal_decomposition(selected_signal, selected_name)

    # ACF and PACF.
    plot_acf_pacf_figures(train)

    p, q, threshold = suggest_orders_from_acf_pacf(
        train,
        max_lag=40,
        max_order=MAX_MODEL_ORDER
    )

    # Save model orders.
    order_df = pd.DataFrame([
        {
            "Selected signal": selected_name,
            "Suggested p from PACF": p,
            "Suggested q from ACF": q,
            "ACF/PACF significance threshold": threshold,
            "Seasonal period used": seasonal_period
        }
    ])

    order_path = os.path.join(OUTPUT_DIR, "q4_selected_orders.csv")
```

```
    order_df.to_csv(order_path, index=False)

    # Fit AR, MA, ARMA, and ARIMA models.
    results = run_forecasting_models(train, test, p, q)

    if len(results) == 0:
        raise RuntimeError("No forecasting model was fitted successfully.")

    # Compare models with RSS.
    comparison_df = save_model_comparison(results)

    best_model = comparison_df.iloc[0]
    print(
        f"\nBest model by RSS: {best_model['Model']} "
        f"with order={best_model['Order']} and RSS={best_model['RSS']:.4f}"
    )

    plot_forecast_comparison(test, results)


if __name__ == "__main__":
    main()
```

q5

```
import os
import itertools
import numpy as np
import pandas as pd


OUTPUT_DIR = "outputs_q5"
os.makedirs(OUTPUT_DIR, exist_ok=True)


try:
    from mlxtend.frequent_patterns import apriori, association_rules
except ImportError:
    apriori = None
    association_rules = None


CSV_PATH = "heart-statlog.csv"

MIN_SUPPORT = 0.25
MIN_LIFT = 1.15
MIN_CONVICTION = 1.5


def binarize_dataset(df):
```

```
    binary = pd.DataFrame(index=df.index)

    binary["age"] = (df["age"] > 50).astype(int)
    binary["sex"] = df["sex"].astype(int)
    binary["chest"] = (df["chest"] > 2.5).astype(int)
    binary["resting_blood_pressure"] = (df["resting_blood_pressure"] > 125).astype(int)
    binary["serum_cholestoral"] = (df["serum_cholestoral"] > 250).astype(int)
    binary["fasting_blood_sugar"] = df["fasting_blood_sugar"].astype(int)
    binary["resting_electrocardiographic_results"] = (
        df["resting_electrocardiographic_results"] != 0
    ).astype(int)
    binary["maximum_heart_rate_achieved"] = (
        df["maximum_heart_rate_achieved"] > 140
    ).astype(int)
    binary["exercise_induced_angina"] = df["exercise_induced_angina"].astype(int)
    binary["oldpeak"] = (df["oldpeak"] != 0).astype(int)
    binary["slope"] = (df["slope"] != 1).astype(int)
    binary["number_of_major_vessels"] = (df["number_of_major_vessels"] != 0).astype(int)
    binary["thal"] = (df["thal"] != 3).astype(int)
    binary["class"] = df["class"].map({"absent": 0, "present": 1}).astype(int)

    return binary


def encode_transactions(binary):
    item_names = {
        "age": {0: "age<=50", 1: "age>50"},
        "sex": {0: "sex=0", 1: "sex=1"},
        "chest": {0: "chest<=2.5", 1: "chest>2.5"},
        "resting_blood_pressure": {
            0: "resting_blood_pressure<=125",
            1: "resting_blood_pressure>125"
        },
        "serum_cholestoral": {
            0: "serum_cholestoral<=250",
            1: "serum_cholestoral>250"
        },
        "fasting_blood_sugar": {
            0: "fasting_blood_sugar=0",
            1: "fasting_blood_sugar=1"
        },
        "resting_electrocardiographic_results": {
            0: "resting_electrocardiographic_results=0",
            1: "resting_electrocardiographic_results!=0"
        },
        "maximum_heart_rate_achieved": {
            0: "maximum_heart_rate_achieved<=140",
            1: "maximum_heart_rate_achieved>140"
        },
        "exercise_induced_angina": {
            0: "exercise_induced_angina=0",
            1: "exercise_induced_angina=1"
```

```
        },
        "oldpeak": {0: "oldpeak=0", 1: "oldpeak!=0"},
        "slope": {0: "slope=1", 1: "slope!=1"},
        "number_of_major_vessels": {
            0: "number_of_major_vessels=0",
            1: "number_of_major_vessels!=0"
        },
        "thal": {0: "thal=3", 1: "thal!=3"},
        "class": {0: "class=absent", 1: "class=present"}
    }

    transaction_df = pd.DataFrame(index=binary.index)

    for column in binary.columns:
        for value, item_name in item_names[column].items():
            transaction_df[item_name] = binary[column].eq(value)

    return transaction_df


def local_apriori(transaction_df, min_support, use_colnames=True):
    n_rows = len(transaction_df)
    columns = list(transaction_df.columns)
    frequent_rows = []
    current_level = []

    for column in columns:
        itemset = frozenset([column])
        support = transaction_df[column].sum() / n_rows

        if support >= min_support:
            frequent_rows.append({"support": support, "itemsets": itemset})
            current_level.append(itemset)

    k = 2

    while current_level:
        candidates = set()

        for left, right in itertools.combinations(current_level, 2):
            candidate = left | right

            if len(candidate) == k:
                subsets_are_frequent = all(
                    frozenset(subset) in current_level
                    for subset in itertools.combinations(candidate, k - 1)
                )

                if subsets_are_frequent:
                    candidates.add(candidate)

        next_level = []
```

```
        for candidate in sorted(candidates, key=lambda x: tuple(sorted(x))):
            mask = transaction_df[list(candidate)].all(axis=1)
            support = mask.sum() / n_rows

            if support >= min_support:
                frequent_rows.append({"support": support, "itemsets": candidate})
                next_level.append(candidate)

        current_level = next_level
        k += 1

    frequent_itemsets = pd.DataFrame(frequent_rows)
    return frequent_itemsets


def local_association_rules(frequent_itemsets, metric="lift", min_threshold=1.0):
    support_lookup = {
        frozenset(row["itemsets"]): row["support"]
        for _, row in frequent_itemsets.iterrows()
    }

    rule_rows = []

    for itemset, itemset_support in support_lookup.items():
        if len(itemset) < 2:
            continue

        items = list(itemset)

        for size in range(1, len(items)):
            for antecedent_tuple in itertools.combinations(items, size):
                antecedents = frozenset(antecedent_tuple)
                consequents = itemset - antecedents

                antecedent_support = support_lookup[antecedents]
                consequent_support = support_lookup[consequents]
                confidence = itemset_support / antecedent_support
                lift = confidence / consequent_support
                leverage = itemset_support - antecedent_support * consequent_support

                if confidence == 1:
                    conviction = np.inf
                else:
                    conviction = (1 - consequent_support) / (1 - confidence)

                rule_rows.append({
                    "antecedents": antecedents,
                    "consequents": consequents,
                    "antecedent support": antecedent_support,
                    "consequent support": consequent_support,
                    "support": itemset_support,
```

```
                    "confidence": confidence,
                    "lift": lift,
                    "leverage": leverage,
                    "conviction": conviction
                })

    rules = pd.DataFrame(rule_rows)

    if rules.empty:
        return rules

    return rules[rules[metric] >= min_threshold].reset_index(drop=True)


def itemset_to_string(itemset):
    return ", ".join(sorted(list(itemset)))


def prepare_rules_for_csv(rules):
    csv_rules = rules.copy()
    csv_rules["antecedents"] = csv_rules["antecedents"].apply(itemset_to_string)
    csv_rules["consequents"] = csv_rules["consequents"].apply(itemset_to_string)
    return csv_rules


def run_association_analysis(transaction_df):
    if apriori is not None and association_rules is not None:
        frequent_itemsets = apriori(
            transaction_df,
            min_support=MIN_SUPPORT,
            use_colnames=True
        )
        rules = association_rules(
            frequent_itemsets,
            metric="lift",
            min_threshold=MIN_LIFT
        )
    else:
        frequent_itemsets = local_apriori(
            transaction_df,
            min_support=MIN_SUPPORT,
            use_colnames=True
        )
        rules = local_association_rules(
            frequent_itemsets,
            metric="lift",
            min_threshold=MIN_LIFT
        )

    rules = rules.sort_values(
        by=["lift", "conviction"],
        ascending=False
```

```
    ).reset_index(drop=True)

    conviction_rules = rules[
        rules["conviction"] > MIN_CONVICTION
    ].sort_values(
        by=["conviction", "lift"],
        ascending=False
    ).reset_index(drop=True)

    return frequent_itemsets, rules, conviction_rules


def filter_disease_rules(rules):
    disease_items = {"class=present", "class=absent"}

    disease_rules = rules[
        rules["consequents"].apply(lambda x: len(set(x) & disease_items) > 0)
    ].copy()

    disease_rules = disease_rules.sort_values(
        by=["conviction", "lift"],
        ascending=False
    ).reset_index(drop=True)

    return disease_rules


def main():
    df = pd.read_csv(CSV_PATH)

    binary = binarize_dataset(df)
    transaction_df = encode_transactions(binary)

    binary.to_csv(os.path.join(OUTPUT_DIR, "q5_binarized_data.csv"), index=False)
    transaction_df.to_csv(os.path.join(OUTPUT_DIR, "q5_transaction_data.csv"),
index=False)

    frequent_itemsets, rules, conviction_rules = run_association_analysis(transaction_df)
    disease_rules = filter_disease_rules(conviction_rules)

    frequent_itemsets_csv = frequent_itemsets.copy()
    frequent_itemsets_csv["itemsets"] =
frequent_itemsets_csv["itemsets"].apply(itemset_to_string)
    frequent_itemsets_csv = frequent_itemsets_csv.sort_values(
        by="support",
        ascending=False
    ).reset_index(drop=True)

    frequent_itemsets_csv.to_csv(
        os.path.join(OUTPUT_DIR, "q5_frequent_itemsets.csv"),
        index=False
    )
```

```
    prepare_rules_for_csv(rules).to_csv(
        os.path.join(OUTPUT_DIR, "q5_lift_rules.csv"),
        index=False
    )
    prepare_rules_for_csv(conviction_rules).to_csv(
        os.path.join(OUTPUT_DIR, "q5_conviction_rules.csv"),
        index=False
    )
    prepare_rules_for_csv(disease_rules).to_csv(
        os.path.join(OUTPUT_DIR, "q5_disease_rules.csv"),
        index=False
    )

    print("Dataset shape:")
    print(df.shape)
    print("\nFrequent itemsets:")
    print(frequent_itemsets_csv.head(15))
    print(f"\nNumber of frequent itemsets: {len(frequent_itemsets_csv)}")
    print(f"Number of rules with lift >= {MIN_LIFT}: {len(rules)}")
    print(f"Number of rules with conviction > {MIN_CONVICTION}: {len(conviction_rules)}")

    print("\nTop conviction rules:")
    print(
        prepare_rules_for_csv(conviction_rules)
        .head(10)[["antecedents", "consequents", "support", "confidence", "lift",
"conviction"]]
    )

    print("\nTop disease-related conviction rules:")
    print(
        prepare_rules_for_csv(disease_rules)
        .head(10)[["antecedents", "consequents", "support", "confidence", "lift",
"conviction"]]
    )


if __name__ == "__main__":
    main()
```